

23 clases documentadas PHP 8 · sin dependencias

Generado 25/08/2026 con phpDocumentor

Referencia técnica generada directamente desde los bloques `/** ... */` del código fuente en `api/`. Cuatro namespaces, cada clase con sus propiedades, métodos, tipos y parámetros tal y como están declarados.

CONTROLLERS

Rutas HTTP

- AlertsController
- LinesController
- RealtimeController
- SearchController
- StopsController
- TimetableController

CORE

Motor compartido

- Cache
- Config
- Database
- Http
- Request
- Response
- Router

MODELS

Acceso a datos

- LineModel
- Search
- ServiceJourney

Stop

SERVICES

Cientes externos

Calendar

MetroAlertsClient

RealtimeMatcher

ScheduleTextClient

SiriAlertsClient

SiriVehicleMonitoringClient

Referencia de la API

Documentación de clases del backend compartido por BizkaiBus+ y Metro+ — `Controllers/` resuelve las rutas HTTP, `Core/` es el motor común (router, request/response, caché, acceso a base de datos), `Models/` consulta el SQLite generado desde GTFS, y `Services/` habla con las fuentes externas (SIRI de Bizkaibus, avisos de Metro Bilbao). Extraída automáticamente del código fuente real; cualquier cambio en los comentarios PHPDoc del proyecto se reflejará aquí en la próxima regeneración.

[↑ Volver al índice](#)

CONTROLLERS

AlertsController

in package APPLICATION

AlertsController.php : 11

Table of Contents

Methods

M `index() : void`

Avisos activos de la red, ruta /alerts. Bifurca por Config::current()['network']: MetroAlertsClient para metro, que devuelve la lista global del CMS de Metro Bilbao sin filtro por línea, porque el metro no tiene ese concepto; y SiriAlertsClient para bus, que consume SIRI-SX de Bizkaibus y admite filtrar por ?line=. Sin ese parámetro, devuelve todos los avisos.

Methods

`index()`

AlertsController.php : 20

Avisos activos de la red, ruta /alerts. Bifurca por Config::current()['network']: MetroAlertsClient para metro, que devuelve la lista global del CMS de Metro Bilbao sin filtro por línea, porque el metro no tiene ese concepto; y SiriAlertsClient para bus, que consume SIRI-SX de Bizkaibus y admite filtrar por ?line=. Sin ese parámetro, devuelve todos los avisos.

PUBLIC

`index(Request $request) : void`

PARAMETERS

`$request` : Request

↑ Volver al índice

CONTROLLERS

LinesController

in package APPLICATION

Table of Contents

Methods

M `index()` : void

Catálogo completo de líneas de la red activa, ruta /lines.

M `scheduleText()` : void

Horario oficial vigente en texto libre, ruta /lines/{id}/schedule-text.

M `show()` : void

Detalle de una línea, ruta /lines/{id}, incluyendo sus journey_patterns (variantes de recorrido). El frontend necesita esos patrones para pintar el mapa de la línea y el selector de sentido antes de pedir un horario.

Methods

`index()`

LinesController.php : 14

Catálogo completo de líneas de la red activa, ruta /lines.

PUBLIC

`index(Request $request) : void`

PARAMETERS

`$request` : Request

`scheduleText()`

LinesController.php : 45

Horario oficial vigente en texto libre, ruta /lines/{id}/schedule-text.

PUBLIC

```
scheduleText(Request $request,  
array<string|int, mixed> $params) : void
```

Solo aplica a bus. Es un scrape del endpoint legado de Bizkaibus (GetLineasHorarios): no viene del GTFS ni está estructurado por parada, así que se devuelve tal cual, como bloques de texto de referencia.

PARAMETERS

\$request : Request

\$params : array<string|int, mixed>

show()

LinesController.php : 26

Detalle de una línea, ruta /lines/{id}, incluyendo sus journey_patterns (variantes de recorrido). El frontend necesita esos patrones para pintar el mapa de la línea y el selector de sentido antes de pedir un horario.

```
PUBLIC  
show(Request $request, array<string|int,  
mixed> $params) : void
```

PARAMETERS

\$request : Request

\$params : array<string|int, mixed>

↑ Volver al índice

CONTROLLERS

RealtimeController

in package APPLICATION

RealtimeController.php : 19

"Detalle del bus": muestra lo que hay realmente en los datos, es decir flota, retraso, parada actual y paradas restantes. Nada de modelo ni amenities inventados: si SIRI no lo publica, la app no lo muestra.

Table of Contents

Methods

M `lineLive()` : void

"En vivo ahora" de una línea: la ruta (paradas ordenadas de cada patrón, para dibujar en el mapa) más los vehículos activos ahora mismo. La posición del vehículo es su última parada confirmada, porque el feed no da GPS continuo, solo parada más orden. El marcador va siempre en una parada real, nunca interpolado entre medias.

M `vehicle()` : void

Detalle en vivo de un vehículo concreto, ruta /vehicles/{tripKey}: su recorrido completo con la hora programada de cada parada, más el enriquecido en tiempo real cuando SIRI-VM lo confirma en circulación.

Methods

`lineLive()`

RealtimeController.php : 28

"En vivo ahora" de una línea: la ruta (paradas ordenadas de cada patrón, para dibujar en el mapa) más los vehículos activos ahora mismo. La posición del vehículo es su última parada confirmada, porque el feed no da GPS continuo, solo parada más orden. El marcador va siempre en una parada real, nunca interpolado entre medias.

PUBLIC

```
lineLive(Request $request,  
array<string|int, mixed> $params) : void
```

PARAMETERS

`$request` : Request

```
$params : array<string|int, mixed>
```

vehicle()

RealtimeController.php : 102

Detalle en vivo de un vehículo concreto, ruta /vehicles/{tripKey}: su recorrido completo con la hora programada de cada parada, más el enriquecido en tiempo real cuando SIRI-VM lo confirma en circulación.

```
PUBLIC
        vehicle(Request $request, array<string|int,
mixed> $params) : void
```

Las paradas que el vehículo ya dejó atrás no llevan ETA, marcadas con isPast a partir del order confirmado por el propio feed.

PARAMETERS

\$request : Request

\$params : array<string|int, mixed>

[↑ Volver al índice](#)

CONTROLLERS

SearchController

in package APPLICATION

SearchController.php : 12

Table of Contents

Methods

M `search()` : void

Búsqueda combinada de paradas y líneas, ruta /search. Exige al menos dos caracteres para no lanzar consultas LIKE contra toda la tabla en cada pulsación del buscador. El hint "hacia X" solo se añade en bus (ver addDirectionHints); en metro se deja explícitamente a null.

M `addDirectionHints()` : array<int, array{id: int, name: string, ar...

Destinos reales que pasan por cada parada, como pista de dirección en la lista de resultados. Se calcula siempre, no solo cuando dos paradas comparten nombre y zona exacta: antes solo se calculaba para esos casos ambiguos, así que la mayoría de paradas con nombre único nunca mostraban hacia dónde iban.

Methods

`search()`

SearchController.php : 20

Búsqueda combinada de paradas y líneas, ruta /search. Exige al menos dos caracteres para no lanzar consultas LIKE contra toda la tabla en cada pulsación del buscador. El hint "hacia X" solo se añade en bus (ver addDirectionHints); en metro se deja explícitamente a null.

PUBLIC

```
search(Request $request) : void
```

PARAMETERS

`$request` : Request

`addDirectionHints()`

SearchController.php : 63

Destinos reales que pasan por cada parada, como pista de dirección en la lista de resultados. Se calcula siempre, no solo cuando dos paradas comparten nombre y zona exacta: antes solo se calculaba para esos casos ambiguos, así que la mayoría de paradas con nombre único nunca mostraban hacia dónde iban.

PRIVATE

```
addDirectionHints(Stop $stopModel,  
array<int, array{id: int, name: string, area: string, lat:  
float, lon: float}> $stops) : array<int, array{id: int, name:  
string, area: string, lat: float, lon: float, hint: ?string}>
```

PARAMETERS

\$stopModel : Stop

\$stops : array<int, array{id: int, name: string, area: string,
lat: float, lon: float}>

Return values

array<int, array{id: int, name: string, area: string, lat: float, lon: float, hint:
?string}>

[↑ Volver al índice](#)

CONTROLLERS

StopsController

in package APPLICATION

StopsController.php : 15

Table of Contents

Methods

M departures() : void

Próximas salidas de una parada, ruta /stops/{id}/departures: la pantalla principal de la app. Enriquece el horario programado con SIRI-VM cuando la red lo tiene (bus); en metro se queda en "scheduled".

M show() : void

Ficha de una parada/estación, ruta /stops/{id}, con las líneas que la sirven.

M tripStops() : void

Secuencia completa de paradas/estaciones de un trayecto programado, con la hora de paso por cada una, sin tiempo real. Es el equivalente de RealtimeController::vehicle() para redes sin SIRI (Metro Bilbao): en vez de "dónde está el vehículo ahora", responde "por dónde pasa este trayecto hasta llegar a mi parada". El stopId de referencia llega por query param ? stopId= y marca esa fila con isTarget=true.

Methods

departures()

StopsController.php : 39

Próximas salidas de una parada, ruta /stops/{id}/departures: la pantalla principal de la app. Enriquece el horario programado con SIRI-VM cuando la red lo tiene (bus); en metro se queda en "scheduled".

PUBLIC

```
departures(Request $request,  
array<string|int, mixed> $params) : void
```

Cuando la config trae direction_reference_stop_id (Abando en Metro+), cada fila lleva además 'direction' para agrupar por sentido de circulación, como un panel físico de andén.

PARAMETERS

\$request : Request

\$params : array<string|int, mixed>

show()

StopsController.php : 18

Ficha de una parada/estación, ruta /stops/{id}, con las líneas que la sirven.

PUBLIC

```
show(Request $request, array<string|int,  
mixed> $params) : void
```

PARAMETERS

\$request : Request

\$params : array<string|int, mixed>

tripStops()

StopsController.php : 130

Secuencia completa de paradas/estaciones de un trayecto programado, con la hora de paso por cada una, sin tiempo real. Es el equivalente de `RealtimeController::vehicle()` para redes sin SIRI (Metro Bilbao): en vez de "dónde está el vehículo ahora", responde "por dónde pasa este trayecto hasta llegar a mi parada". El `stopId` de referencia llega por query param ?`stopId=` y marca esa fila con `isTarget=true`.

PUBLIC

```
tripStops(Request $request,  
array<string|int, mixed> $params) : void
```

PARAMETERS

\$request : Request

\$params : array<string|int, mixed>

[↑ Volver al índice](#)

CONTROLLERS

TimetableController

in package APPLICATION

Table of Contents

Methods

 `show()` : void

Horario completo de una línea para un día y rango horario dados, ruta /lines/{id}/timetable. A diferencia de las salidas "en vivo" de StopsController::departures(), aquí se listan todas las salidas programadas del rango pedido, no solo las próximas, pensado para la pantalla "Consultar Horarios". Si la fecha consultada es hoy, enriquece con SIRI-VM (solo bus); si es otro día, todo queda como "scheduled".

 `hmToSeconds()` : int

"HH:MM" -> segundos desde medianoche. Formato inválido devuelve 0.

Methods

`show()`

TimetableController.php : 25

Horario completo de una línea para un día y rango horario dados, ruta /lines/{id}/timetable. A diferencia de las salidas "en vivo" de StopsController::departures(), aquí se listan todas las salidas programadas del rango pedido, no solo las próximas, pensado para la pantalla "Consultar Horarios". Si la fecha consultada es hoy, enriquece con SIRI-VM (solo bus); si es otro día, todo queda como "scheduled".

PUBLIC

```
show(Request $request, array<string|int,  
mixed> $params) : void
```

PARAMETERS

`$request` : Request

`$params` : array<string|int, mixed>

`hmToSeconds()`

TimetableController.php : 119

"HH:MM" -> segundos desde medianoche. Formato inválido devuelve 0.

PRIVATE

hmToSeconds(string \$hm) : int

PARAMETERS

\$hm : string

Return values

int

[↑ Volver al índice](#)

CORE

Cache

in package APPLICATION

Cache.php : 10

Caché de fichero mínima en el temp del sistema (en Vercel /tmp es la única ruta con permiso de escritura en runtime; funciona igual en local).

Nunca es la fuente de verdad: todo caller debe poder recalcular si falla.

Table of Contents

Methods

M remember() : mixed

Devuelve el valor cacheado bajo \$key si existe y no ha caducado; si no, ejecuta \$producer(), guarda el resultado y lo devuelve. Usado por los clientes de Services/ para no

golpear SIRI ni el CMS de Metro Bilbao en cada petición.

Methods

remember()

`Cache.php` : 18

Devuelve el valor cacheado bajo \$key si existe y no ha caducado; si no, ejecuta \$producer(), guarda el resultado y lo devuelve. Usado por los clientes de Services/ para no golpear SIRI ni el CMS de Metro Bilbao en cada petición.

```
PUBLIC
static remember(string $key,
int $ttlSeconds, callable $producer) : mixed
```

PARAMETERS

\$key : string

\$ttlSeconds : int

\$producer : callable

[↑ Volver al índice](#)

CORE

Config

in package APPLICATION

`Config.php` : 12

Resuelve qué config (y por tanto qué red: bus o metro) está activa para el request actual. Se fija una única vez al arrancar (ver api/index.php) para que

Database::connection() y los controllers compartidos (StopsController, TimetableController) usen siempre la misma red durante todo el request: nunca se decide dos veces ni se mezcla.

Table of Contents

Properties

P  `$current` : array<string|int, mixed>|null

Methods

M  `current()` : array<string|int, mixed>

Config activa; si nadie llamó a set() todavía, arranca en 'bus' por defecto.

M  `set()` : void

Carga y fija la config de la red indicada ('bus' o 'metro') para todo el request.

Properties

`$current`

Config.php : 14

```
    PRIVATE
    static      array<string|int, mixed>|null
    $current
    = null
```

Methods

`current()`

Config.php : 28

Config activa; si nadie llamó a set() todavía, arranca en 'bus' por defecto.

PUBLIC

static

current() : array<string|int, mixed>

Return values

array<string|int, mixed>

set()

Config.php : 17

Carga y fija la config de la red indicada ('bus' o 'metro') para todo el request.

PUBLIC

static

set(string \$network) : void

PARAMETERS

\$network : string

[↑ Volver al indice](#)

CORE

Database

in package APPLICATION

Database.php : 5

Table of Contents

Properties

P `$connections` : array<string|int, mixed>

Una conexión PDO cacheada por red (bus/metro). Un request solo usa una, pero ambas pueden coexistir entre requests dentro del mismo proceso PHP-FPM/CLI-server.

Methods

M `connection()` : PDO

Conexión PDO de solo lectura al SQLite de la red actual, creándola la primera vez.

Properties

`$connections`

Database.php : 12

Una conexión PDO cacheada por red (bus/metro). Un request solo usa una, pero ambas pueden coexistir entre requests dentro del mismo proceso PHP-FPM/CLI-server.

```
    PRIVATE
    static    array<string|int, mixed>
    $connections
    = []
```

Methods

`connection()`

Database.php : 15

Conexión PDO de solo lectura al SQLite de la red actual, creándola la primera vez.

```
    PUBLIC
    static    connection() : PDO
```

Return values

PDO
.....

[↑ Volver al índice](#)

CORE

Http

in package APPLICATION

Http.php : 5

Table of Contents

Methods

M `get()` : string

GET simple vía cURL, usado por todos los clientes de Services/ (SIRI, avisos de Metro Bilbao, horario legado de Bizkaibus). Centraliza el timeout y el manejo de error para no repetir la misma configuración de cURL en cada cliente. Cualquier fallo de red o HTTP >=400 lanza excepción, dejando que cada cliente decida si lo captura o lo propaga.

Methods

get()

Http.php : 14

GET simple vía cURL, usado por todos los clientes de Services/ (SIRI, avisos de Metro Bilbao, horario legado de Bizkaibus). Centraliza el timeout y el manejo de error para no repetir la misma configuración de cURL en cada cliente. Cualquier fallo de red o HTTP >=400 lanza excepción, dejando que cada cliente decida si lo captura o lo propaga.

```
PUBLIC static get(string $url[, int $timeoutSeconds  
= 8 ]) : string
```

PARAMETERS

\$url : string

\$timeoutSeconds : int = 8

Return values

string

↑ Volver al índice

CORE

Request

in package APPLICATION

Request.php : 5

Table of Contents

Properties

- P** \$method : string
- P** \$path : string
- P** \$query : array<string, string>
- P** \$jsonBody : array<string, mixed>|null
- P** \$jsonBodyParsed : bool

Methods

- M** __construct() : mixed

- M `cookie() : string|null`
 - M `json() : array<string, mixed>`
 - M `query() : string|null`
 - M `queryInt() : int|null`
-

Properties

`$method`

Request.php : 7

```
    PUBLIC
        string
    $method
```

`$path`

Request.php : 8

```
    PUBLIC
        string
    $path
```

`$query`

Request.php : 10

```
    PUBLIC
        array<string, string>
```

\$query

\$jsonBody

Request.php : 12

```
    PRIVATE
        array<string, mixed>|null
    $jsonBody
    = null
```

\$jsonBodyParsed

Request.php : 13

```
    PRIVATE
        bool
    $jsonBodyParsed
    = false
```

Methods

__construct()

Request.php : 15

```
    PUBLIC
        __construct() : mixed
```

cookie()

Request.php : 83

PUBLIC

`cookie(string $name) : string|null`

PARAMETERS

`$name : string`

Return values

`string|null`

json()

Request.php : 65

PUBLIC

`json() : array<string, mixed>`

Return values

`array<string, mixed>`

query()

Request.php : 48

PUBLIC

`query(string $key[, string|null $default = null ]) : string|null`

PARAMETERS

`$key : string`

```
$default : string|null = null
```

Return values

`string|null`

queryInt()

Request.php : 56

PUBLIC

```
queryInt(string $key[, int|null $default =  
null ]) : int|null
```

PARAMETERS

`$key` : string

`$default` : int|null = null

Return values

`int|null`

↑ Volver al índice

CORE

Response

in package APPLICATION

Response.php : 5

Table of Contents

Methods

M error() : void

Atajo para respuestas de error, mismo formato que json() con {"error": ...}.

M json() : void

Salida JSON estándar de toda la API: todos los controllers terminan aquí.

Methods

error()

Response.php : 16

Atajo para respuestas de error, mismo formato que json() con {"error": ...}.

```
PUBLIC static error(string $message[, int $status = 400 ]) : void
```

PARAMETERS

\$message : string

\$status : int = 400

json()

Response.php : 8

Salida JSON estándar de toda la API: todos los controllers terminan aquí.

```
PUBLIC static json(mixed $data[, int $status = 200 ]) : void
```

PARAMETERS

\$data : mixed

\$status : int = 200

[↑ Volver al índice](#)

CORE

Router

in package APPLICATION

Router.php : 5

Table of Contents

Properties

 `$routes` : array<int, array{method: string, pattern: string, rege...

Methods

 `delete()` : void

Registra una ruta DELETE.

 `dispatch()` : void

Encuentra la ruta cuyo patrón case con request->path y method, y delega en su handler, capturando cualquier excepción del handler como 500 en vez de dejar pasar un fatal sin respuesta JSON. Distingue "no existe ninguna ruta con este path" (404) de "existe pero con otro method" (405), comprobando el regex por separado del method.

 `get()` : void

Registra una ruta GET, ej: \$router->get('/stops/{id}', [...]).

 `post()` : void

Registra una ruta POST.

 `add()` : void

Compila un patrón tipo '/stops/{id}/departures' a una regex con grupos de captura, guardando el nombre de cada {parámetro} en orden para poder reconstruir un array asociativo tras el match en dispatch().

Properties

\$routes

Router.php : 8

```
    PRIVATE
        array<int, array{method: string, pattern: string,
regex: string, params: string[], handler: callable}>
    $routes
    = []
```

Methods

delete()

Router.php : 23

Registra una ruta DELETE.

```
    PUBLIC
        delete(string $pattern, callable $handler)
    : void
```

PARAMETERS

\$pattern : string

\$handler : callable

dispatch()

Router.php : 57

Encuentra la ruta cuyo patrón case con request->path y method, y delega en su handler, capturando cualquier excepción del handler como 500 en vez de dejar pasar un fatal sin respuesta JSON. Distingue "no existe ninguna ruta con este path" (404) de "existe pero con otro method" (405), comprobando el regex por separado del method.

PUBLIC

```
dispatch(Request $request) : void
```

PARAMETERS

\$request : Request

get()

Router.php : 11

Registra una ruta GET, ej: \$router->get('/stops/{id}', [...]).

PUBLIC

```
get(string $pattern, callable $handler) :  
void
```

PARAMETERS

\$pattern : string

\$handler : callable

post()

Router.php : 17

Registra una ruta POST.

PUBLIC

```
post(string $pattern, callable $handler) :  
void
```

PARAMETERS

\$pattern : string

\$handler : callable

add()

Router.php : 33

Compila un patrón tipo '/stops/{id}/departures' a una regex con grupos de captura, guardando el nombre de cada {parámetro} en orden para poder reconstruir un array asociativo tras el match en dispatch().

PRIVATE

```
add(string $method, string $pattern,  
callable $handler) : void
```

PARAMETERS

\$method : string

\$pattern : string

\$handler : callable

[↑ Volver al índice](#)

MODELS

LineModel

in package APPLICATION

LineModel.php : 5

Table of Contents

Properties

 \$pdo : PDO

Methods

 __construct() : mixed

- M `all()` : `array<int, array{id: int, code: string, name: string}>`
 - M `find()` : `array<string|int, mixed>|null`
 - M `patterns()` : `array<int, array{id: string, headsign: ?string}>`
 - M `patternsWithStops()` : `array<int, array{id: string, headsign: ?st...`
 - M `search()` : `array<int, array{id: int, code: string, name: string}>`
-

Properties

\$pdo

LineModel.php : 7

```
    PRIVATE
    PDO
    $pdo
```

Methods

__construct()

LineModel.php : 7

PUBLIC

```
__construct(PDO $pdo) : mixed
```

PARAMETERS

`$pdo` : PDO

all()

LineModel.php : 23

PUBLIC

```
all() : array<int, array{id: int, code: string, name: string}>
```

Return values

```
array<int, array{id: int, code: string, name: string}>
```

find()

LineModel.php : 11

PUBLIC

```
find(int $id) : array<string|int, mixed>|null
```

PARAMETERS

`$id` : int

Return values

```
array<string|int, mixed>|null
```

patterns()

LineModel.php : 44

PUBLIC

```
patterns(int $lineId) : array<int, array{id: string, headsign: ?string}>
```

PARAMETERS

`$lineId` : int

Return values

array<int, array{id: string, headsign: ?string}> —

patrones/direcciones distintos de una línea

patternsWithStops()

LineModel.php : 52

PUBLIC

```
patternsWithStops(int $lineId) : array<int,  
array{id: string, headsign: ?string, stops: array}>
```

PARAMETERS

\$lineId : int

Return values

array<int, array{id: string, headsign: ?string, stops: array}>

search()

LineModel.php : 29

PUBLIC

```
search(string $query[, int $limit = 10 ]) :  
array<int, array{id: int, code: string, name: string}>
```

PARAMETERS

\$query : string

\$limit : int = 10

Return values

array<int, array{id: int, code: string, name: string}>

[↑ Volver al índice](#)

MODELS

Search

in package APPLICATION

Search.php : 6

Tiene que ser igual que normalize() de scripts/build-database.php para que las búsquedas casen con name_normalized.

Table of Contents

Methods

M normalize() : string

Methods

normalize()

Search.php : 8

PUBLIC

static

normalize(string \$text) : string

PARAMETERS

\$text : string

Return values

string

[↑ Volver al índice](#)

MODELS

ServiceJourney

in package APPLICATION

ServiceJourney.php : 7

Table of Contents

Constants

 **DEDUPE_TOLERANCE_SECONDS** : mixed = 90

Igual que el gap de clustering del build (scripts/build-database.php): dos variantes de calendario para el mismo (línea, trip_number) que en esta parada caen a menos de esto una de otra son el mismo viaje real, no dos salidas distintas.

 **LAST_STOP_NAME_SUBQUERY** : mixed = '(SELECT s2.name FROM passing...

El headsign de journey_patterns viene del trip_headsign del GTFS, que no siempre coincide con la última parada real del recorrido concreto.

 **PAST_GRACE_SECONDS** : mixed = 900

Un bus retrasado puede seguir sin llegar mucho después de su hora programada. Si el margen hacia atrás fuera pequeño, se excluiría de aquí (por hora programada) antes de que el enriquecido en vivo tenga ocasión de comprobar si en realidad sigue en camino, y desaparecería de la app justo cuando estuviera "LLEGANDO" sin haber llegado todavía.

Properties

 **\$pdo** : PDO

Methods

 **__construct()** : mixed

 **arrivalSecondsAtOrder()** : int|null

Hora programada de llegada en una posición concreta (seq_order) de un viaje. Permite a RealtimeMatcher calcular "tiempo programado restante desde la última parada confirmada" en vez de solo "hora original + retraso plano".

M `findByLineAndTrip()` : `array<string|int, mixed>|null`

Un viaje concreto por su clave (línea, número de trip, primera salida), tal como llega en tripKey desde el frontend.

M `stopsForJourney()` : `array<int, array{seq_order: int, stop_id: in...`

Recorrido completo de un viaje, parada a parada en orden, con hora programada de llegada y salida en cada una. Base de las pantallas de "detalle de trayecto" y del modal sin tiempo real de metro.

M `timetableForLine()` : `array<int, array<string, mixed>>`

Salidas programadas desde la parada de origen de una línea, para el día de la semana en que cae \$date, dentro de un rango horario. Usado en "Tabla Horaria".

M `upcomingAtStop()` : `array<int, array<string, mixed>>`

Salidas programadas en una parada, desde ahora en adelante, deduplicadas entre variantes de calendario que coinciden en trip_number/hora (ver README sobre por qué más de un calendario puede casar con el mismo día).

M  `dedupeByTrip()` : `array<string|int, mixed>`

Colapsa filas del mismo (line_id, trip_number) cuya hora de salida cae dentro de DEDUPE_TOLERANCE_SECONDS: son variantes de calendario que casan con el mismo día para el mismo viaje real, no dos salidas distintas (ver esa constante). Conserva la primera fila de cada grupo, en el orden en que ya vinieron ordenadas por la consulta SQL.

Constants

DEDUPE_TOLERANCE_SECONDS

ServiceJourney.php : 12

Igual que el gap de clustering del build (scripts/build-database.php): dos variantes de calendario para el mismo (línea, trip_number) que en esta parada caen a menos de esto una de otra son el mismo viaje real, no dos salidas distintas.

PRIVATE

`mixed`

DEDUPE_TOLERANCE_SECONDS

`= 90`

LAST_STOP_NAME_SUBQUERY

ServiceJourney.php : 35

El headsign de journey_patterns viene del trip_headsign del GTFS, que no siempre coincide con la última parada real del recorrido concreto.

PRIVATE

mixed

LAST_STOP_NAME_SUBQUERY

```
= '(  
    SELECT s2.name FROM passing_times pt2  
    JOIN stops s2 ON s2.id = pt2.stop_id  
    WHERE pt2.service_journey_id = sj.id  
    ORDER BY pt2.seq_order DESC LIMIT 1  
)'
```

Verificado con datos reales de Metro Bilbao: 172 de 265 patrones (65%) tienen headsign distinto de su última parada, incluyendo casos donde dos trenes con el MISMO headsign salen a la misma hora pero uno de ellos en realidad se queda corto (p.ej. "Etxebarri" a las 06:00 dos veces: uno llega a Etxebarri de verdad, el otro solo hasta Indautxu). En Bizkaibus esta discrepancia es aún más frecuente (95%) pero ahí es esperada: el headsign es el destino comercial anunciado del recorrido, no necesariamente la última parada de cada variante. Por eso este dato se calcula siempre pero cada red decide en el frontend si usarlo (Metro+) o seguir mostrando el headsign de GTFS (bus).

PAST_GRACE_SECONDS

ServiceJourney.php : 19

Un bus retrasado puede seguir sin llegar mucho después de su hora programada. Si el margen hacia atrás fuera pequeño, se excluiría de aquí (por hora programada) antes de que el enriquecido en vivo tenga ocasión de comprobar si en realidad sigue en camino, y desaparecería de la app justo cuando estuviera "LLEGANDO" sin haber llegado todavía.

```
    PRIVATE
        mixed
    PAST_GRACE_SECONDS
    = 900
```

Properties

\$pdo

ServiceJourney.php : 42

```
    PRIVATE
        PDO
    $pdo
```

Methods

__construct()

ServiceJourney.php : 42

```
    PUBLIC
        __construct(PDO $pdo) : mixed
```

PARAMETERS

\$pdo : PDO

arrivalSecondsAtOrder()

ServiceJourney.php : 214

Hora programada de llegada en una posición concreta (seq_order) de un viaje. Permite a RealtimeMatcher calcular "tiempo programado restante desde la última parada confirmada" en vez de solo "hora original + retraso plano".

PUBLIC

```
arrivalSecondsAtOrder(string $serviceJourneyId, int $seqOrder)  
: int|null
```

PARAMETERS

\$serviceJourneyId : string

\$seqOrder : int

Return values

int|null

findByLineAndTrip()

ServiceJourney.php : 189

Un viaje concreto por su clave (línea, número de trip, primera salida), tal como llega en tripKey desde el frontend.

PUBLIC

```
findByLineAndTrip(int $lineId,  
string $tripNumber, int $firstDepartureSeconds) :  
array<string|int, mixed>|null
```

PARAMETERS

\$lineId : int

\$tripNumber : string

\$firstDepartureSeconds : int

Return values

array<string|int, mixed>|null

stopsForJourney()

ServiceJourney.php : 235

Recorrido completo de un viaje, parada a parada en orden, con hora programada de llegada y salida en cada una. Base de las pantallas de "detalle de trayecto" y del modal sin tiempo real de metro.

PUBLIC

```
stopsForJourney(string $serviceJourneyId) :  
array<int, array{seq_order: int, stop_id: int, name: string,  
arrival_seconds: int, departure_seconds: int}>
```

PARAMETERS

\$serviceJourneyId : string

Return values

array<int, array{seq_order: int, stop_id: int, name: string, arrival_seconds: int, departure_seconds: int}>

timetableForLine()

ServiceJourney.php : 142

Salidas programadas desde la parada de origen de una línea, para el día de la semana en que cae \$date, dentro de un rango horario. Usado en "Tabla Horaria".

PUBLIC

```
timetableForLine(int $lineId,  
DateTime $date, int $hourFromSeconds, int $hourToSeconds) :  
array<int, array<string, mixed>>
```

PARAMETERS

\$lineId : int

```
$date : DateTime
$hourFromSeconds : int
$hourToSeconds : int
```

Return values

```
array<int, array<string, mixed>>
```

upcomingAtStop()

ServiceJourney.php : 66

Salidas programadas en una parada, desde ahora en adelante, deduplicadas entre variantes de calendario que coinciden en trip_number/hora (ver README sobre por qué más de un calendario puede casar con el mismo día).

PUBLIC

```
upcomingAtStop(int $stopId[, int $limit = 8
][, int $windowSeconds = 4 * 3600 ][, int|null $referenceStopId
= null ]) : array<int, array<string, mixed>>
```

\$referenceStopId, cuando se pasa, añade una columna "direction" a cada fila: 'toward_reference' si esta parada va antes que la de referencia en la secuencia del propio trayecto (o esa parada no aparece en el trayecto en absoluto, ver más abajo), 'away_from_reference' si va después. Pensado para Metro+ (referencia = Abando, el centro real de la red) para agrupar las salidas por sentido de circulación en dos columnas, como un panel físico de andén. Verificado con datos reales que comparar el seq_order de la parada consultada contra el de Abando dentro del MISMO journey_pattern predice el sentido con fiabilidad total en los patrones que pasan por ambas. No tiene sentido para bus (sin concepto de "centro" único de red), así que el parámetro es opcional y por defecto no calcula nada.

PARAMETERS

```
$stopId : int
$limit : int = 8
$windowSeconds : int = 4 * 3600
```

```
$referenceStopId : int|null = null
```

Return values

```
array<int, array<string, mixed>>
```

dedupeByTrip()

ServiceJourney.php : 255

Colapsa filas del mismo (line_id, trip_number) cuya hora de salida cae dentro de DEDUPE_TOLERANCE_SECONDS: son variantes de calendario que casan con el mismo día para el mismo viaje real, no dos salidas distintas (ver esa constante). Conserva la primera fila de cada grupo, en el orden en que ya vinieron ordenadas por la consulta SQL.

PRIVATE

```
dedupeByTrip(array<string|int,  
mixed> $rows, int $limit) : array<string|int, mixed>
```

PARAMETERS

```
$rows : array<string|int, mixed>
```

```
$limit : int
```

Return values

```
array<string|int, mixed>
```

[↑ Volver al índice](#)

MODELS

Stop

in package APPLICATION

Table of Contents

Properties

P \$pdo : PDO

Methods

M __construct() : mixed

M find() : array<string|int, mixed>|null

Ficha básica de una parada/estación por id, sin las líneas que la sirven (ver linesServing).

M headsignsFor() : array<int, string>

Destinos reales (headsigns de patrón) que pasan por esta parada, hasta \$limit valores distintos. Usado como pista para diferenciar paradas que comparten nombre y zona exacta (p.ej. dos andenes de la misma marquesina, uno de ida y otro de vuelta) en los resultados de búsqueda.

M linesServing() : array<int, array{id: int, code: string, name: s...

M search() : array<int, array{id: int, name: string, area: string,...

*Búsqueda de paradas/estaciones por nombre o zona, normalizando el texto de entrada (sin tildes, minúsculas) contra las columnas *_normalized ya precalculadas por el ETL, para que un usuario que escriba "bilbo" o "getxo" sin acentos siga encontrando resultados.*

M safeQuery() : PDOStatement

Prueba primero la consulta con `area`; si esa columna aún no existe (p.ej. data/bizkaibus.sqlite se generó antes de añadir la geocodificación, ver scripts/build-database.php), cae a la versión sin ella.

Properties

\$pdo

Stop.php : 7

PRIVATE
PDO
\$pdo

Methods

`__construct()`

Stop.php : 7

PUBLIC

```
__construct(PDO $pdo) : mixed
```

PARAMETERS

`$pdo` : PDO

`find()`

Stop.php : 12

Ficha básica de una parada/estación por id, sin las líneas que la sirven (ver `linesServing`).

PUBLIC

```
find(int $id) : array<string|int,  
mixed>|null
```

PARAMETERS

`$id` : `int`

Return values

`array<string|int, mixed>|null`

`headsigsFor()`

Stop.php : 59

Destinos reales (headsigs de patrón) que pasan por esta parada, hasta \$limit valores distintos. Usado como pista para diferenciar paradas que

comparten nombre y zona exacta (p.ej. dos andenes de la misma marquesina, uno de ida y otro de vuelta) en los resultados de búsqueda.

```
PUBLIC
        headsignsFor(int $stopId[, int $limit = 2
]) : array<int, string>
```

PARAMETERS

`$stopId : int`

`$limit : int = 2`

Return values

`array<int, string>`

linesServing()

Stop.php : 75

```
PUBLIC
        linesServing(int $stopId) : array<int,
array{id: int, code: string, name: string}>
```

PARAMETERS

`$stopId : int`

Return values

`array<int, array{id: int, code: string, name: string}> —`

líneas que pasan por esta parada

search()

Stop.php : 34

*Búsqueda de paradas/estaciones por nombre o zona, normalizando el texto de entrada (sin tildes, minúsculas) contra las columnas *_normalized ya precalculadas por el ETL, para que un usuario que escriba "bilbo" o "getxo" sin acentos siga encontrando resultados.*

PUBLIC

```
search(string $query[, int $limit = 10 ]) :  
array<int, array{id: int, name: string, area: string, lat:  
float, lon: float}>
```

PARAMETERS

\$query : string

\$limit : int = 10

Return values

array<int, array{id: int, name: string, area: string, lat: float, lon: float}>

safeQuery()

Stop.php : 94

Prueba primero la consulta con `area`; si esa columna aún no existe (p.ej. data/bizkaibus.sqlite se generó antes de añadir la geocodificación, ver scripts/build-database.php), cae a la versión sin ella.

PRIVATE

```
safeQuery(string $sql, string $fallbackSql,  
array<string|int, mixed> $args[, array<string|int,  
mixed>|null $fallbackArgs = null ]) : PDOStatement
```

PARAMETERS

\$sql : string

\$fallbackSql : string

\$args : array<string|int, mixed>

\$fallbackArgs : array<string|int, mixed>|null = null

Return values

[↑ Volver al índice](#)

SERVICES

Calendar

in package APPLICATION

Calendar.php : 5

Table of Contents

Methods

M nowSecondsSinceMidnight() : int

M secondsToHm() : string

Segundos desde medianoche (puede superar 86400 en viajes que cruzan medianoche) a "HH:MM".

M todayMadrid() : DateTime

M todayWeekdayBit() : int

Día de la semana ISO (1=lunes..7=domingo) como bit de máscara, igual que `service_calendars.weekday_mask`.

M weekdayBitFor() : int

Methods

nowSecondsSinceMidnight()

Calendar.php : 12

PUBLIC

static

nowSecondsSinceMidnight() : int

Return values

int

secondsToHm()

Calendar.php : 31

Segundos desde medianoche (puede superar 86400 en viajes que cruzan medianoche) a "HH:MM".

PUBLIC

static

secondsToHm(int \$seconds) : string

PARAMETERS

\$seconds : int

Return values

string

todayMadrid()

Calendar.php : 7

PUBLIC

static

todayMadrid() : DateTime

Return values

DateTime

todayWeekdayBit()

Calendar.php : 19

Día de la semana ISO (1=lunes..7=domingo) como bit de máscara, igual que `service_calendars.weekday_mask`.

PUBLIC

static todayWeekdayBit() : int

Return values

int

weekdayBitFor()

Calendar.php : 24

PUBLIC

static weekdayBitFor(DateTime \$date) : int

PARAMETERS

\$date : DateTime

Return values

int

↑ Volver al índice

SERVICES

MetroAlertsClient

in package APPLICATION

MetroAlertsClient.php : 17

Avisos/incidencias de Metro Bilbao (endpoint JSON propio del CMS, no SIRI-SX como Bizkaibus, que no publica ese formato). El station_id que trae cada aviso pertenece al sistema interno del CMS, no al stop_id del GTFS público, y no hay forma fiable de cruzarlos, así que los avisos se muestran como lista global de la red, no filtrados por estación. El propio título del aviso ya suele nombrar la estación en texto libre (p.ej.

"Ascensor exterior de Areeta fuera de servicio").

Table of Contents

Properties

 `$config` : array<string|int, mixed>

Methods

 `__construct()` : mixed

 `fetchAlerts()` : array<int, array{summary: string, description: s...

Avisos activos ahora mismo, cacheados según metro_alerts.cache_ttl_seconds.

 `parse()` : array<string|int, mixed>

Filtra el JSON crudo del CMS a los avisos publicados y aún no finalizados.

Properties

\$config

MetroAlertsClient.php : 19

```
    PRIVATE
        array<string|int, mixed>
    $config
```

Methods

__construct()

MetroAlertsClient.php : 21

PUBLIC

```
__construct(array<string|int,  
mixed> $config) : mixed
```

PARAMETERS

\$config : array<string|int, mixed>

fetchAlerts()

MetroAlertsClient.php : 33

Avisos activos ahora mismo, cacheados según metro_alerts.cache_ttl_seconds.

PUBLIC

```
fetchAlerts() : array<int, array{summary:  
string, description: string, startTime: ?string, endTime: ?  
string}>
```

Si el CMS de Metro Bilbao falla o no responde, devuelve lista vacía en vez de propagar el error: un aviso caído no debe tumbar el resto de la app.

Return values

array<int, array{summary: string, description: string, startTime: ?string, endTime: ?string}>

parse()

MetroAlertsClient.php : 47

Filtra el JSON crudo del CMS a los avisos publicados y aún no finalizados.

PRIVATE

```
static parse(string $body) :  
array<string|int, mixed>
```

PARAMETERS

\$body : string

Return values

array<string|int, mixed>

[↑ Volver al índice](#)

SERVICES

RealtimeMatcher

in package APPLICATION

RealtimeMatcher.php : 13

Enriquece las salidas programadas con datos en vivo de SIRI-VM cuando los hay. Agrupa por (line_id, trip_number) y busca el departure_seconds más cercano dentro de un margen; ver SiriVehicleMonitoringClient para por qué ni el id exacto ni la hora exacta funcionan contra este feed.

Table of Contents

Constants

 MATCH_TOLERANCE_SECONDS : mixed = 180

 POSITION_SANITY_SECONDS : mixed = 15 * 60

Si la estimación por posición y la de retraso plano difieren más de esto, es que el Order del feed en vivo no encaja con nuestra secuencia de paradas (verificado: pasa ~1 de cada 9 veces), así que se descarta y se usa el retraso plano.

 STALE_GRACE_SECONDS : mixed = 120

Properties

- P** `$journeyModel` : `ServiceJourney|null`
- P** `$vmMap` : `array<string|int, mixed>`

Methods

- M** `__construct()` : `mixed`
- M** `enrich()` : `array<int, array<string, mixed>>`
- M** `etaForStop()` : `array{0: int, 1: int}`

Prefiere "ahora + tiempo programado restante desde la última parada confirmada del bus" antes que "hora programada original + retraso plano reportado", porque se ancla a dónde se vio al bus la última vez en vez de fiarse de una única cifra de retraso para todo el trayecto restante. Si la estimación por posición no está disponible o difiere demasiado, cae al retraso plano. Pública para que RealtimeController la reutilice por parada en "Detalle del bus".

- M** `lookup()` : `array<string|int, mixed>|null`

Candidato en vivo más cercano para (line_id, trip_number) dentro del margen de \$firstDepartureSeconds, o null.

Constants

MATCH_TOLERANCE_SECONDS

`RealtimeMatcher.php` : 16

```
PRIVATE
    mixed
    MATCH_TOLERANCE_SECONDS
    = 180
```

POSITION_SANITY_SECONDS

`RealtimeMatcher.php` : 20

Si la estimación por posición y la de retraso plano difieren más de esto, es que el Order del feed en vivo no encaja con nuestra secuencia de

paradas (verificado: pasa ~1 de cada 9 veces), así que se descarta y se usa el retraso plano.

```
PRIVATE
    mixed
    POSITION_SANITY_SECONDS
    = 15 * 60
```

STALE_GRACE_SECONDS

RealtimeMatcher.php : 15

```
PRIVATE
    mixed
    STALE_GRACE_SECONDS
    = 120
```

Properties

\$journeyModel

RealtimeMatcher.php : 23

```
PRIVATE
    ServiceJourney|null
    $journeyModel
```

\$vmMap

RealtimeMatcher.php : 22

```
        PRIVATE
            array<string|int, mixed>
$vmMap
```

Methods

__construct()

RealtimeMatcher.php : 25

```
        PUBLIC
            __construct(array<string|int,
mixed> $vmMap[, ServiceJourney|null $journeyModel = null ]) :
mixed
```

PARAMETERS

\$vmMap : array<string|int, mixed>

\$journeyModel : ServiceJourney|null = null

enrich()

RealtimeMatcher.php : 36

```
        PUBLIC
            enrich(array<int, array<string,
mixed>> $rows) : array<int, array<string, mixed>>
```

PARAMETERS

\$rows : array<int, array<string, mixed>>

cada una debe traer line_id, trip_number, first_departure_seconds, arrival_seconds y (para poder calcular ETA por posición)

service_journey_id

Return values

array<int, array<string, mixed>> —

las mismas filas más
status/delaySeconds/etaSeconds/vehicleRef/currentStopId

etaForStop()

RealtimeMatcher.php : 99

Prefiere "ahora + tiempo programado restante desde la última parada confirmada del bus" antes que "hora programada original + retraso plano reportado", porque se ancla a dónde se vio al bus la última vez en vez de fiarse de una única cifra de retraso para todo el trayecto restante. Si la estimación por posición no está disponible o difiere demasiado, cae al retraso plano. Pública para que RealtimeController la reutilice por parada en "Detalle del bus".

PUBLIC

```
etaForStop(string|null $serviceJourneyId,  
int $targetArrivalSeconds, array<string|int, mixed>|null $live)  
: array{0: int, 1: int}
```

PARAMETERS

\$serviceJourneyId : string|null

\$targetArrivalSeconds : int

\$live : array<string|int, mixed>|null

Return values

array{0: int, 1: int} —

[etaSeconds, delaySecondsToDisplay]

lookup()

RealtimeMatcher.php : 123

Candidato en vivo más cercano para (line_id, trip_number) dentro del margen de \$firstDepartureSeconds, o null.

PUBLIC

```
lookup(int $lineId, string $tripNumber,  
int $firstDepartureSeconds) : array<string|int, mixed>|null
```

PARAMETERS

\$lineId : int

\$tripNumber : string

\$firstDepartureSeconds : int

Return values

array<string|int, mixed>|null

[↑ Volver al índice](#)

SERVICES

ScheduleTextClient

in package APPLICATION

ScheduleTextClient.php : 18

Endpoint legado GetLineasHorarios: horario en texto libre ("Sabados y festivos: de 10:35 a 12:35 cada hora"), no por parada. Se muestra como referencia complementaria de solo lectura junto al horario estructurado (derivado del GTFS), nunca se parsea a passing_times.

Sus códigos de línea van con ceros a la izquierda (p.ej. "A0651") a diferencia de los nuestros ("A651"), así que el match quita los no-dígitos y compara el número de línea pelado.

Table of Contents

Properties

 `$config` : array<string|int, mixed>

Methods

 `__construct()` : mixed

 `fetchForLine()` : array<int, array<string, string>>

 `fetchAll()` : array<int, array<int, array<string, string>>>

 `parse()` : array<string|int, mixed>

Properties

`$config`

ScheduleTextClient.php : 20

PRIVATE

array<string|int, mixed>

`$config`

Methods

`__construct()`

ScheduleTextClient.php : 22

PUBLIC

```
        __construct(array<string|int,  
mixed> $config) : mixed
```

PARAMETERS

\$config : array<string|int, mixed>

fetchForLine()

ScheduleTextClient.php : 28

PUBLIC

```
        fetchForLine(int $lineId) : array<int,  
array<string, string>>
```

PARAMETERS

\$lineId : int

Return values

array<int, array<string, string>> —

bloques campo=>texto para la línea dada, vacío si no hay

fetchAll()

ScheduleTextClient.php : 38

PRIVATE

```
        fetchAll() : array<int, array<int,  
array<string, string>>>
```

Return values

array<int, array<int, array<string, string>>> —

indexado por id numérico de línea pelado

parse()

ScheduleTextClient.php : 51

```
PRIVATE static parse(string $xmlString) :  
array<string|int, mixed>
```

PARAMETERS

\$xmlString : string

Return values

array<string|int, mixed>

[↑ Volver al índice](#)

SERVICES

SiriAlertsClient

in package APPLICATION

SiriAlertsClient.php : 12

Alertas de servicio SIRI-SX (bizkaibus-service-alerts.xml). Feed pequeño (~180KB), se pide en vivo y se cachea poco tiempo: nunca es estático.

Table of Contents

Properties

P `$config` : array<string|int, mixed>

Methods

M `__construct()` : mixed

M `alertsByLine()` : array<string, array<int, array{summary: string,...

Las mismas alertas de fetchAlerts(), reagrupadas por line_id de línea afectada, para que AlertsController pueda filtrar por ?line= sin volver a pedir el feed.

M `fetchAlerts()` : array<int, array{summary: string, description: s...

Todas las alertas activas del feed, cacheadas según siri.cache_ttl_seconds.

M `parse()` : array<string|int, mixed>

Traduce el XML SIRI-SX (PtSituationElement) a la forma plana que usa el resto de la app.

M `textByLang()` : string

Properties

`$config`

SiriAlertsClient.php : 14

PRIVATE

array<string|int, mixed>

`$config`

Methods

`__construct()`

SiriAlertsClient.php : 16

PUBLIC

`__construct(array<string|int, mixed> $config) : mixed`

PARAMETERS

`$config : array<string|int, mixed>`

alertsByLine()

SiriAlertsClient.php : 47

Las mismas alertas de fetchAlerts(), reagrupadas por line_id de línea afectada, para que AlertsController pueda filtrar por ?line= sin volver a pedir el feed.

PUBLIC

```
alertsByLine() : array<string, array<int,  
array{summary: string, description: string, startTime: ?string,  
endTime: ?string}>>
```

Return values

```
array<string, array<int, array{summary: string, description: string,  
startTime: ?string, endTime: ?string}>>
```

fetchAlerts()

SiriAlertsClient.php : 27

Todas las alertas activas del feed, cacheadas según siri.cache_ttl_seconds.

PUBLIC

```
fetchAlerts() : array<int, array{summary:  
string, description: string, startTime: ?string, endTime: ?  
string, lineRefs: string[]}>
```

Si SIRI-SX falla, devuelve lista vacía en vez de propagar el error.

Return values

```
array<int, array{summary: string, description: string, startTime: ?string,
endTime: ?string, lineRefs: string[]}>
```

parse()

SiriAlertsClient.php : 64

Traduce el XML SIRI-SX (PtSituationElement) a la forma plana que usa el resto de la app.

```
PRIVATE
static parse(string $xmlString) :
array<string|int, mixed>
```

PARAMETERS

\$xmlString : string

Return values

array<string|int, mixed>

textByLang()

SiriAlertsClient.php : 115

```
PRIVATE
static textByLang(mixed $elements,
string $lang) : string
```

PARAMETERS

\$elements : mixed

\$lang : string

Return values

string

[↑ Volver al índice](#)

SERVICES

SiriVehicleMonitoringClient

in package APPLICATION

SiriVehicleMonitoringClient.php : 29

Feed SIRI-VM "vehicle monitoring" (mal nombrado bizkaibus-trip-updates.xml en origen). Da retraso en vivo + parada/orden actual por cada viaje activo.

Los VehicleJourneyRef (p.ej. "trp_A3513_907_OP44LV_61500_O44LV3513_351302_6") NO coinciden exactamente con el id de service_journeys del estático. Verificado que solo los tokens de línea y trip_number son fiables: el token de calendario y todo lo que va después usan un vocabulario distinto en vivo que en estático. trip_number tampoco es único por sí solo: es un id de bloque/vehículo reutilizado en decenas de salidas distintas a lo largo del día (verificado: el trip_number "1137" de la línea 2322 aparece a las 06:23, 07:36, 08:36, 09:33... todo el día). El token de segundos de salida que lleva incrustado lo acota, pero tampoco coincide byte a byte con el horario estático. Verificado con pares reales que, por ejemplo, un "72900" en vivo contra un "72928" estático para la misma (línea, trip) son la misma salida física, con decenas de segundos de diferencia. Por eso se agrupa por (line_id, trip_number) y se elige el first_departure_seconds más cercano dentro de un margen, en vez de exigir igualdad exacta en ningún sitio.

Table of Contents

Properties

 \$config : array<string|int, mixed>

Methods

M `__construct()` : mixed

M `fetchActiveTrips()` : array<string, array<int, array{departureSec...

Todos los vehículos en circulación ahora mismo según SIRI-VM, cacheados según `siri.cache_ttl_seconds`. Si el feed falla, devuelve mapa vacío en vez de propagar el error: sin datos en vivo, la app sigue funcionando con el horario programado.

M `parse()` : array<string|int, mixed>

Extrae `line_id` y `trip_number` del `VehicleJourneyRef` y agrupa las actividades del feed por esa clave.

M `parseIsoDuration()` : int

Parsea el subconjunto de duraciones ISO 8601 que usa este feed (PT0S, PT5M, PT1H2M, con signo).

Properties

`$config`

SiriVehicleMonitoringClient.php : 31

PRIVATE

array<string|int, mixed>
`$config`

Methods

`__construct()`

SiriVehicleMonitoringClient.php : 33

PUBLIC

`__construct(array<string|int,
mixed> $config) : mixed`

PARAMETERS

```
$config : array<string|int, mixed>
```

fetchActiveTrips()

SiriVehicleMonitoringClient.php : 46

Todos los vehículos en circulación ahora mismo según SIRI-VM, cacheados según siri.cache_ttl_seconds. Si el feed falla, devuelve mapa vacío en vez de propagar el error: sin datos en vivo, la app sigue funcionando con el horario programado.

PUBLIC

```
fetchActiveTrips() : array<string,  
array<int, array{departureSeconds: int, delaySeconds: int,  
vehicleRef: string, currentStopId: ?int, order: ?int}>>
```

Return values

array<string, array<int, array{departureSeconds: int, delaySeconds: int, vehicleRef: string, currentStopId: ?int, order: ?int}>> —

indexado por "{lineId}|{tripNumber}"

parse()

SiriVehicleMonitoringClient.php : 60

Extrae line_id y trip_number del VehicleJourneyRef y agrupa las actividades del feed por esa clave.

PRIVATE

```
static parse(string $xmlString) :  
array<string|int, mixed>
```

PARAMETERS

\$xmlString : string

Return values

`array<string|int, mixed>`

parseIsoDuration()

SiriVehicleMonitoringClient.php : 116

Parsea el subconjunto de duraciones ISO 8601 que usa este feed (PT0S, PT5M, PT1H2M, con signo).

PRIVATE

`static parseIsoDuration(string $iso) : int`

PARAMETERS

`$iso` : string

Return values

`int`